

Neggaoui Reda
BUT GEII

MISSION DE STAGE

Développement d'un générateur automatique de microcode TILLYS

Refonte d'un outil existant (Python, Flask, HTMX, PyInstaller)

Sommaire

1. Point de départ : le code du collègue	2
2. Choix techniques et architecture	3
3. Réalisation et fonctionnalités	4
4. Difficultés techniques rencontrées	5
5. Bilan	6

1. Point de départ : le code du collègue

Ma mission a débuté par l'analyse d'un fichier `Microcode.html` développé par un collègue de l'agence. Il s'agissait d'un fichier unique de **1 091 lignes** regroupant dans un seul document HTML l'interface graphique, la logique de configuration et la génération du microcode, entièrement réalisées en JavaScript.

Ce que le fichier existant contenait

- Interface à deux panneaux : configuration à gauche, microcode généré à droite
- Modales d'ajout d'UTL et d'accès (lecteurs classiques et serrures APERIO)
- Génération du microcode TILLYS et du fichier de propriétés serveur en JavaScript
- Export et import de la configuration en JSON
- Coloration syntaxique du microcode affiché

Les limites identifiées

- **Impossible de modifier un accès** après l'avoir créé, seule la suppression était disponible
- **Tout en JavaScript** : logique de l'application, rendu, état ... , difficile à faire évoluer pour une équipe Python
- **Aucune structure** : un seul fichier de 1 091 lignes sans organisation claire du code ni commentaires
- **Distribution complexe** : dépendait d'un environnement Python spécifique et d'un script PowerShell lié à un chemin local

Décision : Conserver la logique de génération et le design visuel existants comme base de travail, et reconstruire entièrement le reste en Python, de manière propre, structurée et facilement modifiable.

2. Choix techniques et architecture

La refonte repose sur trois décisions structurantes :

Tout en Python, zéro JavaScript custom

Toute la logique de l'application, mémorisation de la configuration, traitement des actions, génération du code qui sont gérées par Python avec Flask. Les pages affichées dans le navigateur sont construites par Python à chaque action. HTMX permet de mettre à jour une partie de la page sans recharger toute la fenêtre, simplement en ajoutant des attributs dans le HTML, sans écrire une seule ligne de JavaScript.

Séparation claire des responsabilités

Fichier	Rôle unique
sources/lanceur.py	Démarre Flask et ouvre le navigateur (point d'entrée de l'exé)
sources/serveur.py	Toutes les actions disponibles dans l'application, mémorisation des données en cours, actions utilisateur
sources/generateur.py	Génération du microcode TILLYS (traduite depuis le JS d'origine)
interface/pages HTML/	Pages HTML le moteur de pages de Flask (affichage uniquement, aucune logique)
interface/statique/styles.css	Mise en page (inchangée par rapport à l'original)

Exécutable qui tourne sans rien installer via PyInstaller

Le fichier `construire_exe.bat` regroupe l'ensemble du projet en un seul fichier `.exe` d'environ 15-20 Mo contenant Python, Flask et toutes les pages de l'interface. Les collègues n'ont besoin de rien installer, un double-clic suffit. Un point important lors de la compilation : le script de build doit être lancé depuis le dossier `sources/` pour que Python reconnaisse correctement les fichiers du projet.

3. Réalisation et fonctionnalités

3.1 Fonctionnalités reprises et améliorées

Élément du fichier original	Évolution AP traduite
Interface HTML/CSS	Conservée (même design visuel, ajout du bouton )
Logique de génération JS	Portée en Python dans <code>generateur.py</code> (même résultat, meilleure maintenabilité)
Gestion de l'état (JS)	Déplacée en mémoire temporaire de l'application côté Python
Rendu dynamique (JS)	Remplacé par HTMX + le moteur de pages de Flask (aucun JS à écrire)
Export / Import JSON	Réimplémenté côté Python
Script PowerShell de lancement	Remplacé par un exe PyInstaller qui tourne sans rien installer

3.2 Nouvelles fonctionnalités développées

Modification des accès

La fonctionnalité principale ajoutée est la possibilité de **modifier un accès existant**. Un bouton  ouvre le formulaire pré-rempli avec toutes les valeurs actuelles de l'accès. Côté serveur, la action `/acces/valider` détecte si un `acc_id` est présent dans la requête : si oui, elle met à jour l'accès existant (en conservant son ID) ; sinon, elle en crée un nouveau.

Tri automatique par numéro de lecteur

Après chaque ajout ou modification d'accès, la fonction `trier_acces_bus()` réordonne automatiquement les accès d'un bus par numéro de lecteur croissant. Cela correspond à la logique de câblage physique des installations et évite les erreurs de numérotation.

Modale de confirmation personnalisée

Les suppressions d'UTL et d'accès déclenchaient auparavant la popup native du navigateur (`hx-confirm`). Elle a été remplacée par une modale intégrée à l'interface, cohérente avec le design de l'application, avec un message et un titre adaptés à chaque action.

Réinitialisation à chaque ouverture

La session est vidée à chaque ouverture de l'application (`session.clear()` dans la action `GET /`). Cela garantit un état propre à chaque démarrage. La configuration peut être sauvegardée via l'export JSON et rechargée à la prochaine session.

4. Difficultés techniques rencontrées

Compatibilité des chemins exe / script Python

Au premier lancement de l'exé, l'application affichait une erreur `TemplateNotFound: interface.html` et refusait de démarrer. Le problème : en lancement normal Python, les fichiers HTML sont trouvés via leur emplacement dans le dossier du projet. Dans un exe compilé, PyInstaller regroupe tous les fichiers dans un dossier temporaire dont le chemin change à chaque exécution, Flask ne savait donc plus où chercher les pages. La solution a été de détecter automatiquement dans quel mode l'application tourne :

```
if getattr(sys, 'frozen', False):
    racine = Path(sys._MEIPASS)      # Mode exe compilé
else:
    racine = Path(__file__).resolve().parent.parent  # Mode script
```

Un second problème : si le script de compilation était lancé depuis le dossier racine du projet, Python ne reconnaissait plus `serveur.py` et `generateur.py` comme des fichiers de code à exécuter. Il les traitait comme de simples fichiers texte, ce qui causait une erreur au démarrage. Lancer la compilation depuis le dossier `sources/` a résolu ce problème.

Gestion des onglets avec HTMX

Quand l'utilisateur cliquait sur un onglet (Microcode, Propriétés, Résumé), le contenu se mettait bien à jour mais l'onglet restait visuellement surligné sur l'ancien choix, il fallait rafraîchir la page entière pour voir le bon onglet sélectionné. La cause : seul le contenu du panneau était mis à jour, pas les onglets. La solution a été d'indiquer à HTMX de changer l'apparence des onglets immédiatement au moment du clic, avant même que la réponse du serveur revienne.

5. Bilan

Objectif	Résultat
Pouvoir modifier un accès existant	☒ Bouton  avec formulaire pré-rempli
Code maintenable en Python uniquement	☒ Zéro JavaScript custom (tout en Python + le moteur de pages de Flask)
Exe qui tourne sans rien installer distribuable	☒ Fichier .exe unique (~15-20 Mo), aucune installation
Structure claire et documentée	☒ 3 fichiers Python, 7 pages HTML, commentés en français
Tri automatique des accès	☒ Réordonnancement par numéro de lecteur après chaque action
Interface identique à l'original	☒ Même design visuel, modales améliorées

Ce que cette mission m'a apporté : Reprendre un code existant et l'améliorer sans perdre ce qui fonctionnait est très différent de construire quelque chose de zéro. Il a fallu d'abord comprendre la logique du code du collègue, décider ce qui méritait d'être gardé, et tout reconstruire de manière plus claire et plus solide, tout en livrant à la fin un outil réellement utilisé par l'équipe.